第6章 数组

6.1 学习目标

- 理解数组的概念与存储方式
- 掌握一维数组的定义、初始化和遍历
- 掌握二维数组的定义和使用
- 掌握字符数组和字符串处理函数
- 学会常用排序和查找算法

6.2 为什么需要数组

```
// 没有数组：存储5个学生成绩
int s1, s2, s3, s4, s5;
scanf("%d%d%d%d%d", &s1, &s2, &s3, &s4, &s5);

// 有数组：简洁高效
int scores[5];
for (int i = 0; i < 5; i++)
    scanf("%d", &scores[i]);
```

> 数组将一组**相同类型**的数据存储在**连续**的内存空间中，通过下标访问。

6.3 一维数组

6.3.1 定义与初始化

```
// 定义格式：类型 数组名[长度];
int a[5];                    // 定义长度为5的整型数组，值未初始化（垃圾值）

// 初始化方式
int b[5] = {1, 2, 3, 4, 5}; // 全部初始化
int c[5] = {1, 2};           // 部分初始化，剩余自动补0: {1,2,0,0,0}
int d[] = {1, 2, 3};         // 不指定长度，自动推断为3
int e[100] = {0};            // 全部初始化为0
```

6.3.2 数组的访问

```
int arr[5] = {10, 20, 30, 40, 50};

arr[0] = 100;        // 修改第一个元素
printf("%d", arr[2]);  // 访问第三个元素，输出30

// 下标从0开始！长度为n的数组，有效下标为 0 ~ n-1
arr[5] = 60;         // ? 越界！C语言不检查数组越界，后果严重
```

```
内存分布：
arr[0]  arr[1]  arr[2]  arr[3]  arr[4]
  10      20      30      40      50
  ↓
低地址  ←  连续存储  →  高地址
```

6.3.3 数组的遍历

```
#include <stdio.h>
int main()
{
    int arr[5] = {10, 20, 30, 40, 50};

    // 正向遍历
    printf("正向: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    // 反向遍历
    printf("\n反向: ");
    for (int i = 4; i >= 0; i--)
        printf("%d ", arr[i]);

    printf("\n");
    return 0;
}
```

6.3.4 经典案例

案例1：求最大值和最小值

```
#include <stdio.h>
int main()
{
    int arr[] = {34, 12, 56, 78, 23, 9, 45};
    int n = sizeof(arr) / sizeof(arr[0]);  // 计算数组长度
    int max = arr[0], min = arr[0];

    for (int i = 1; i < n; i++)
    {
```

```
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }

    printf("最大值: %d, 最小值: %d\n", max, min);
    return 0;
}
```

案例2：数组逆序

```
#include <stdio.h>
int main()
{
    int arr[] = {1, 2, 3, 4, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    // 两两交换
    for (int i = 0; i < n / 2; i++)
    {
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    // 输出: 6 5 4 3 2 1

    return 0;
}
```

6.4 排序算法

6.4.1 冒泡排序（Bubble Sort）

思想：相邻元素两两比较，大的往后"冒泡"。每轮确定一个最大值。

```
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)          // n-1轮
    {
        for (int j = 0; j < n - 1 - i; j++)  // 每轮比较次数递减
        {
            if (arr[j] > arr[j + 1])         // 升序排列
            {
                // 交换
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
```

```
            }
        }
    }
}

int main()
{
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    printf("排序后: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    // 输出: 11 12 22 25 34 64 90
    return 0;
}
```

```
冒泡排序过程示意:
初始: [64, 34, 25, 12, 22, 11, 90]
第1轮:[34, 25, 12, 22, 11, 64, 90]  ← 90归位
第2轮:[25, 12, 22, 11, 34, 64, 90]  ← 64归位
...
```

6.4.2 选择排序（Selection Sort）

思想：每轮选出最小的元素，放到已排序部分的末尾。

```
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        // 交换
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

6.5 查找算法

6.5.1 顺序查找

```
// 返回目标值下标，未找到返回-1
int sequentialSearch(int arr[], int n, int target)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == target)
            return i;
    }
    return -1;
}
```

6.5.2 二分查找（要求数组有序）

```
int binarySearch(int arr[], int n, int target)
{
    int left = 0, right = n - 1;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;  // 防溢出

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;  // 未找到
}

int main()
{
    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15};
    int n = sizeof(arr) / sizeof(arr[0]);

    int result = binarySearch(arr, n, 7);
    printf("7 的下标是: %d\n", result);  // 4
    return 0;
}
```

6.6 二维数组

6.6.1 定义与初始化

```
// 定义格式：类型 数组名[行数][列数];
int matrix[3][4];                          // 3行4列，未初始化

// 初始化方式
int a[2][3] = {{1,2,3}, {4,5,6}};         // 按行初始化
int b[2][3] = {1,2,3,4,5,6};              // 按顺序初始化（与上面等价）
int c[2][3] = {{1}, {4}};                 // 部分初始化，剩余补0
int d[][3] = {1,2,3,4,5,6};               // 省略行数，自动推断
```

6.6.2 二维数组的访问和遍历

```
#include <stdio.h>
int main()
{
    int arr[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9,10,11,12}
    };

    // 双重循环遍历
    for (int i = 0; i < 3; i++)      // 行
    {
        for (int j = 0; j < 4; j++)  // 列
        {
            printf("%3d ", arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

6.6.3 经典案例

案例1：矩阵转置

```
#include <stdio.h>
int main()
{
    int a[2][3] = {{1,2,3}, {4,5,6}};
    int b[3][2];  // 转置后3行2列

    // 转置: b[j][i] = a[i][j]
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            b[j][i] = a[i][j];
```

```
    // 输出转置结果
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
            printf("%d ", b[i][j]);
        printf("\n");
    }
    return 0;
}
```

案例2：杨辉三角

```
#include <stdio.h>
int main()
{
    int n = 10;
    int triangle[10][10] = {0};

    for (int i = 0; i < n; i++)
    {
        triangle[i][0] = 1;          // 每行第一个为1
        triangle[i][i] = 1;          // 每行最后一个为1

        for (int j = 1; j < i; j++)
        {
            triangle[i][j] = triangle[i-1][j-1] + triangle[i-1][j];
        }
    }

    // 打印
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= i; j++)
            printf("%-4d", triangle[i][j]);
        printf("\n");
    }
    return 0;
}
```

6.7 字符数组与字符串

6.7.1 字符数组的定义

```
char str1[10];                          // 字符数组
char str2[10] = {'H','e','l','l','o'};  // 逐个字符初始化
char str3[10] = "Hello";                // 字符串初始化（自动添加'\0'）
char str4[] = "Hello";                  // 自动分配6个字节（含'\0'）
```

6.7.2 字符串结束标志 '\0'

```
内存布局：  char s[6] = "Hello";
┌───┬───┬───┬───┬───┬───┐
| H | e | l | l | o |\0 |
└───┴───┴───┴───┴───┴───┘
  s[0] s[1] s[2] s[3] s[4] s[5]
```

> **重要**：C语言中字符串以 `'\0'`（ASCII码0）作为结束标志。定义字符数组存储字符串时，长度要比实际字符数多1。没有 `'\0'` 的不是字符串，只是普通字符数组。

6.7.3 字符串输入输出

```c
#include <stdio.h>
int main()
{
    char name[20];

    printf("请输入姓名: ");
    scanf("%s", name);        // 注意：不需要&! 数组名本身就是地址
    printf("你好, %s!\n", name);

    // gets 已废弃，改用 fgets（安全，不会溢出）
    printf("请输入全名: ");
    getchar();                // 消耗上次输入留下的换行符
    fgets(name, 20, stdin);   // 最多读19个字符+'\0'
    printf("全名: %s", name);

    return 0;
}
```

> `scanf("%s", str)` 遇到空格就结束读取，且数组名前**不加** `&`！

6.7.4 常用字符串处理函数（string.h）

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[20] = "Hello";
    char s2[20] = "World";
    char s3[20];

    // strlen: 求字符串长度（不包括'\0'）
    printf("s1长度: %d\n", strlen(s1));   // 5

    // strcpy: 字符串复制
    strcpy(s3, s1);                       // s3 = "Hello"
    printf("s3: %s\n", s3);
```

```
    // strcat: 字符串连接
    strcat(s1, " ");
    strcat(s1, s2);                         // s1 = "Hello World"
    printf("连接后: %s\n", s1);

    // strcmp: 字符串比较（按字典序）
    // 返回值: <0 s1<s2, 0 s1==s2, >0 s1>s2
    if (strcmp(s1, s2) == 0)
        printf("字符串相等\n");
    else
        printf("字符串不相等\n");

    // strchr: 查找字符
    char *p = strchr(s1, 'W');
    if (p != NULL)
        printf("找到W，位置: %d\n", p - s1);

    return 0;
}
```

6.8 综合案例：学生成绩管理系统

```
#include <stdio.h>
#define MAX_STUDENTS 50
#define NAME_LEN 20

int main()
{
    char names[MAX_STUDENTS][NAME_LEN];  // 二维字符数组存姓名
    int scores[MAX_STUDENTS];            // 一维数组存成绩
    int n = 0;                           // 实际学生数

    // 输入
    printf("请输入学生人数: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("请输入第%d个学生的姓名和成绩: ", i + 1);
        scanf("%s %d", names[i], &scores[i]);
    }

    // 计算统计
    int sum = 0, max = scores[0], min = scores[0];
    int maxIndex = 0, minIndex = 0;

    for (int i = 0; i < n; i++)
    {
        sum += scores[i];
        if (scores[i] > max) { max = scores[i]; maxIndex = i; }
        if (scores[i] < min) { min = scores[i]; minIndex = i; }
```

```
    }

    float avg = (float)sum / n;

    // 输出统计
    printf("\n===== 成绩统计 =====\n");
    printf("平均分: %.2f\n", avg);
    printf("最高分: %s (%d分)\n", names[maxIndex], max);
    printf("最低分: %s (%d分)\n", names[minIndex], min);

    // 统计各等级人数
    int countA = 0, countB = 0, countC = 0, countD = 0, countF = 0;
    for (int i = 0; i < n; i++)
    {
        if (scores[i] >= 90) countA++;
        else if (scores[i] >= 80) countB++;
        else if (scores[i] >= 70) countC++;
        else if (scores[i] >= 60) countD++;
        else countF++;
    }
    printf("A(90+): %d人\n", countA);
    printf("B(80-89): %d人\n", countB);
    printf("C(70-79): %d人\n", countC);
    printf("D(60-69): %d人\n", countD);
    printf("F(<60): %d人\n", countF);

    return 0;
}
```

6.9 本章小结

知识点	要点
一维数组	`类型 数组名[长度];` 下标从0开始
数组初始化	`int a[5] = {0};` 全部初始化
数组长度	`sizeof(arr) / sizeof(arr[0])`
冒泡排序	相邻比较交换，O(n?)
二分查找	前提有序，O(log n)
二维数组	`类型 数组名[行][列];`
字符串	以 `'\0'` 结尾的字符数组
常用函数	`strlen, strcpy, strcat, strcmp`

6.10 课后作业

基础练习

1. 选择题

(1) 定义 `int a[10];` 最后一个元素是（ ）：

A. `a[9]`　B. `a[10]`　C. `a[0]`　D. `a[1]`

(2) `char s[10] = "Hello";` 则 `strlen(s)` 的值是（ ）：

A. 6　B. 5　C. 10　D. 不确定

2. 判断题

(1) `int a[5] = {1,2,3,4,5,6};` 是正确的数组定义。（ ）

(2) 数组名是地址常量，不能赋值。（ ）

编程练习

3. 数组去重

输入10个整数，去除重复元素后输出。

4. 插入排序

输入一个有序数组，再输入一个数，将它插入到数组中，保持有序。

5. 矩阵对角线

输入一个3×3矩阵，求主对角线和副对角线上元素之和。

6. 字符串反转

不使用 `strrev` 函数，自己实现字符串反转。

7. 回文字符串判断

输入一个字符串，判断它是否是回文（正读和反读都一样，如 "level"）。

8. 成绩排序

输入10个学生的成绩，按从高到低排序输出（同时输出名次）。

> **学习建议**：数组 = 批量处理数据的基础。排序和查找算法要多敲几遍，理解后才能真正掌握。